

Duale Hochschule Baden-Württemberg Heilbronn

Datenbanken 2

Vorlesungsunterlagen

Agenda

- Organisatorisches
- Einführung: Moderne Datenanwendungen
- Datenformate und semi-strukturierte Daten
- NoSQL-Datenbanken
- Dokumentenorientierte Datenspeicherung mit MongoDB
- Datenverarbeitung und Datenbankbindung mit Python
- Entwicklung von Datenanwendungen mit Streamlit
- Projektarbeit: Entwicklung einer eigenen Datenanwendung
- Präsentation und Diskussion der Projekte

Agenda

- **Organisatorisches**
- Einführung: Moderne Datenanwendungen
- Datenformate und semi-strukturierte Daten
- NoSQL-Datenbanken
- Dokumentenorientierte Datenspeicherung mit MongoDB
- Datenverarbeitung und Datenbankbindung mit Python
- Entwicklung von Datenanwendungen mit Streamlit
- Projektarbeit: Entwicklung einer eigenen Datenanwendung
- Präsentation und Diskussion der Projekte

Ziel der Veranstaltung

In dieser Veranstaltung lernen Sie, wie moderne Datenanwendungen entwickelt werden.

Sie werden:

- mit strukturierten und semi-strukturierten Daten arbeiten
- relationale und NoSQL-Datenbanken einsetzen
- Daten mit Python verarbeiten
- eine eigene Datenanwendung entwickeln

Am Ende der Veranstaltung erstellen Sie eine **eigene Streamlit-Anwendung**, die sowohl an eine MySQL-Datenbank als auch eine MongoDB angebunden ist.

Termine

Räume: C.022 & C.023

- Mi 01.04.2026 13:15-16:30 Uhr
 - Do 02.04.2026 09:00-12:15 Uhr
 - Mi 08.04.2026 13:15-16:30 Uhr
 - Mi 15.04.2026 13:15-16:30 Uhr (**remote**)
 - Mi 22.04.2026 13:15-14:45 Uhr (**remote**)
 - Mi 29.04.2026 13:15-16:30 Uhr (Präsentationen)
-
- Zoom-Raum: Wir nutzen den von WI24A2:
 - <https://eu02web.zoom-x.de/j/9271083935>
 - Passwort: wi24a2

Projekt und Prüfungsleistung

Im Laufe der Veranstaltung entwickeln Sie eine eigene Anwendung.

Rahmenbedingungen:

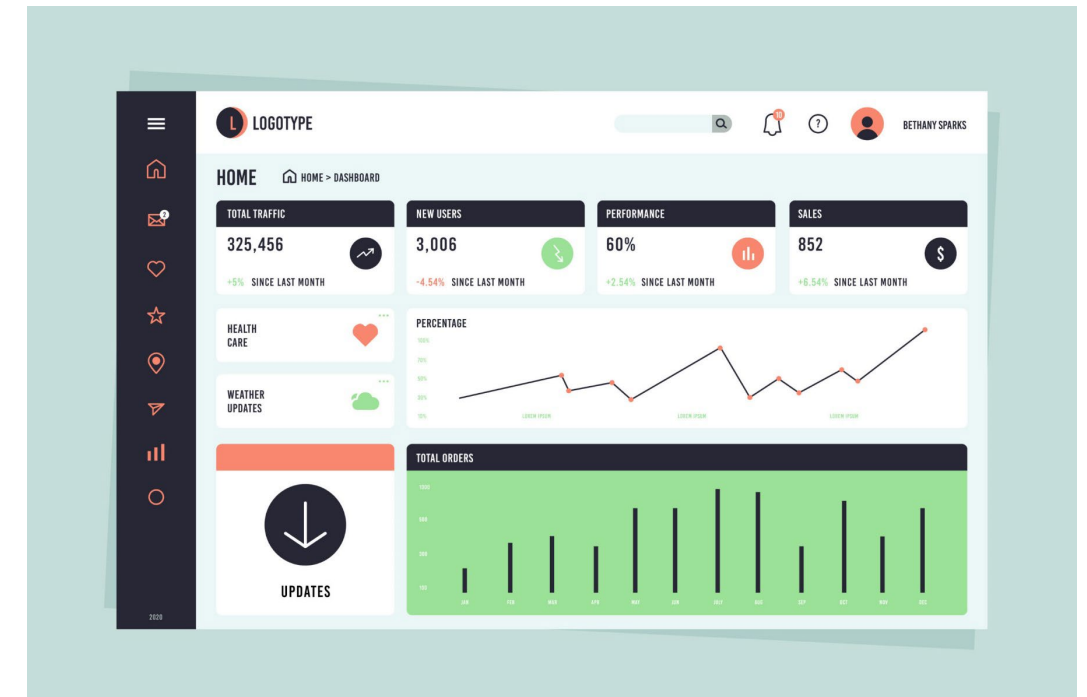
- Nutzung eines öffentlichen Datensatzes (z.B. Kaggle)
- Speicherung der Daten in zwei Datenbanken
- Entwicklung einer interaktiven Anwendung mit **Streamlit**
- Am Ende präsentieren Sie Ihre Anwendung kurz im Kurs

Prüfungsleistung: Assignment

Die Prüfungsleistung besteht aus einem **Projektbericht (5–10 Seiten)**.

Der Bericht beschreibt:

- den verwendeten Datensatz
- das entwickelte Datenmodell
- die technische Umsetzung
- die entwickelte Anwendung

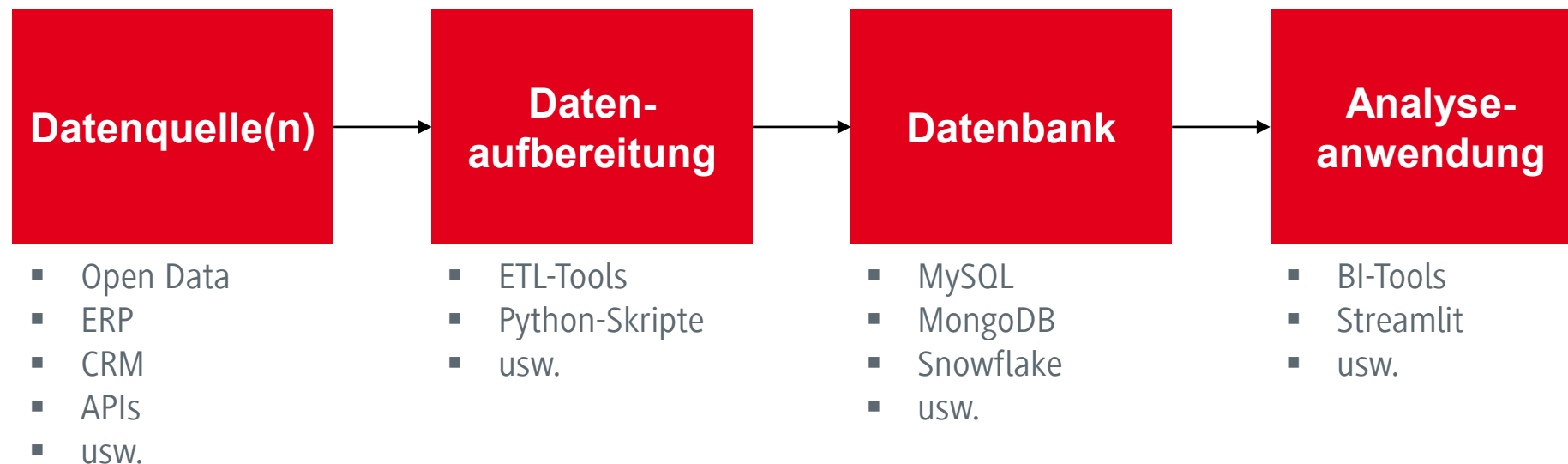


Agenda

- Organisatorisches
- **Einführung: Moderne Datenanwendungen**
- Datenformate und semi-strukturierte Daten
- NoSQL-Datenbanken
- Dokumentenorientierte Datenspeicherung mit MongoDB
- Datenverarbeitung und Datenbankbindung mit Python
- Entwicklung von Datenanwendungen mit Streamlit
- Projektarbeit: Entwicklung einer eigenen Datenanwendung
- Präsentation und Diskussion der Projekte

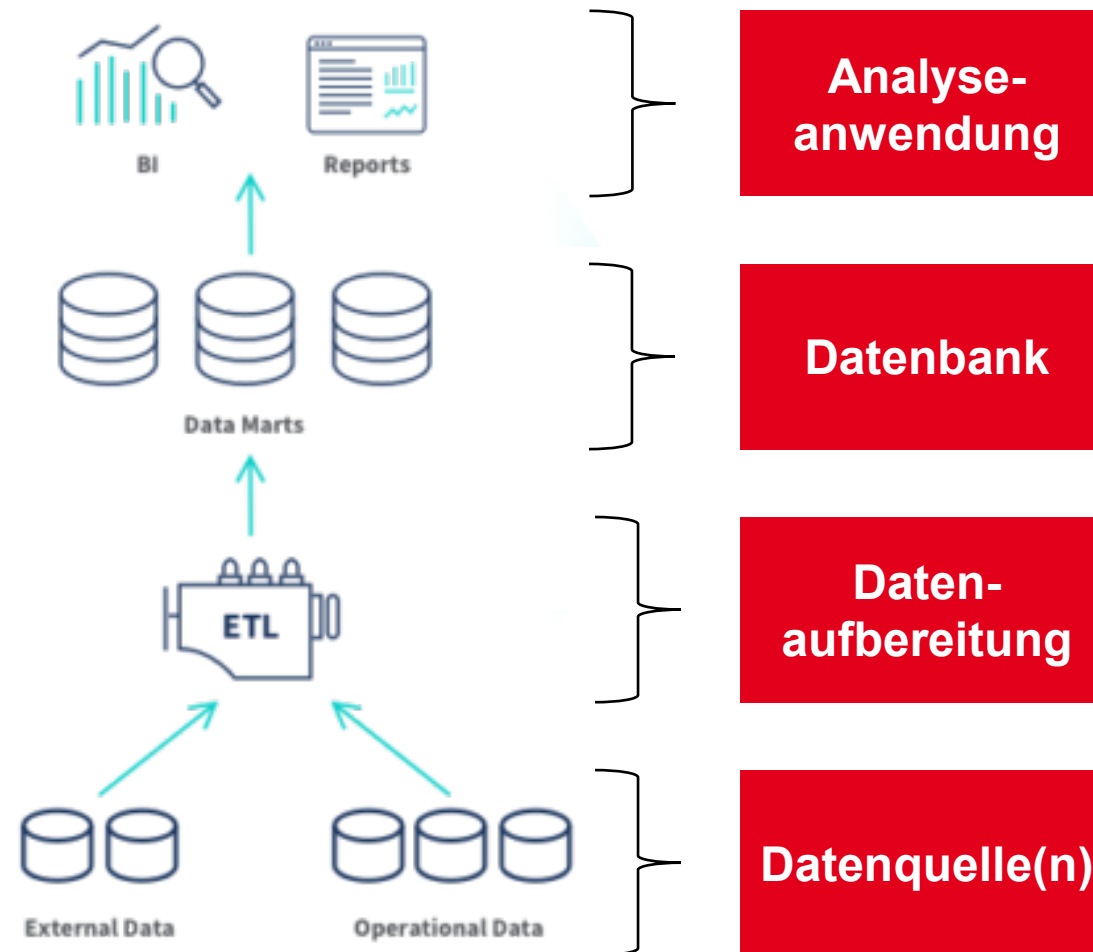
Typische Architektur analytischer Datenanwendungen

- Viele moderne Datenanwendungen dienen der **Analyse und Exploration von Daten**.
- Sie bestehen häufig aus mehreren Komponenten:



Klassisches Beispiel: ETL und Business Intelligence

Data Warehouse



<https://insightsoftware.com/de/blog/remodel-your-oracle-cloud-data-with-a-data-lakehouse/>

Offene Frage in die Runde



Warum nutzen so viele Unternehmen eine separate/zentrale Datenbank („Data Warehouse“) zur Analyse?

Motivation für eine zentrale Datenbasis zur Analyse

Entlastung der operativen Systeme (Performance-Grund)

- Operative Systeme (z.B. ERP, CRM) sind für **schnelle Transaktionen** optimiert (z.B. Bestellungen erfassen, Kunden verwalten).
- Aufwendige **Analysen mit komplexen SQL-Abfragen** hätten die Performance dieser Systeme stark beeinträchtigt.
- Eine zentrale Datenbasis ermöglicht es, **Analysen auf einer separaten Plattform** durchzuführen, ohne die operativen Systeme zu verlangsamen.

Datenintegration aus verschiedenen Quellen

- Unternehmen haben oft **mehrere operative Systeme** (z.B. getrennte Systeme für Vertrieb, Buchhaltung, Lagerverwaltung).
- Daten lagen **fragmentiert und oft in unterschiedlichen Formaten** vor.
- Eine zentrale Datenbasis wurde eingeführt, um **alle relevanten Daten an einem Ort zusammenzuführen** und eine **einheitliche Sicht** auf das Unternehmen zu ermöglichen.

Bessere Datenqualität und Historisierung

- Operative Systeme speichern oft **nur aktuelle Daten**, aber für Analysen benötigt man **historische Daten** (z.B. Umsatztrends über mehrere Jahre).
- Eine zentrale Datenbasis ermöglicht **Datenhistorie, Datenbereinigung und eine einheitliche Datenstruktur** für konsistente Berichte.

Data Warehouse

Ein **Data Warehouse** ist eine zentrale Datenbank, die Daten aus verschiedenen Quellen integriert, um sie für Berichte und Analysen bereitzustellen.

Gründe für ihre Entstehung:

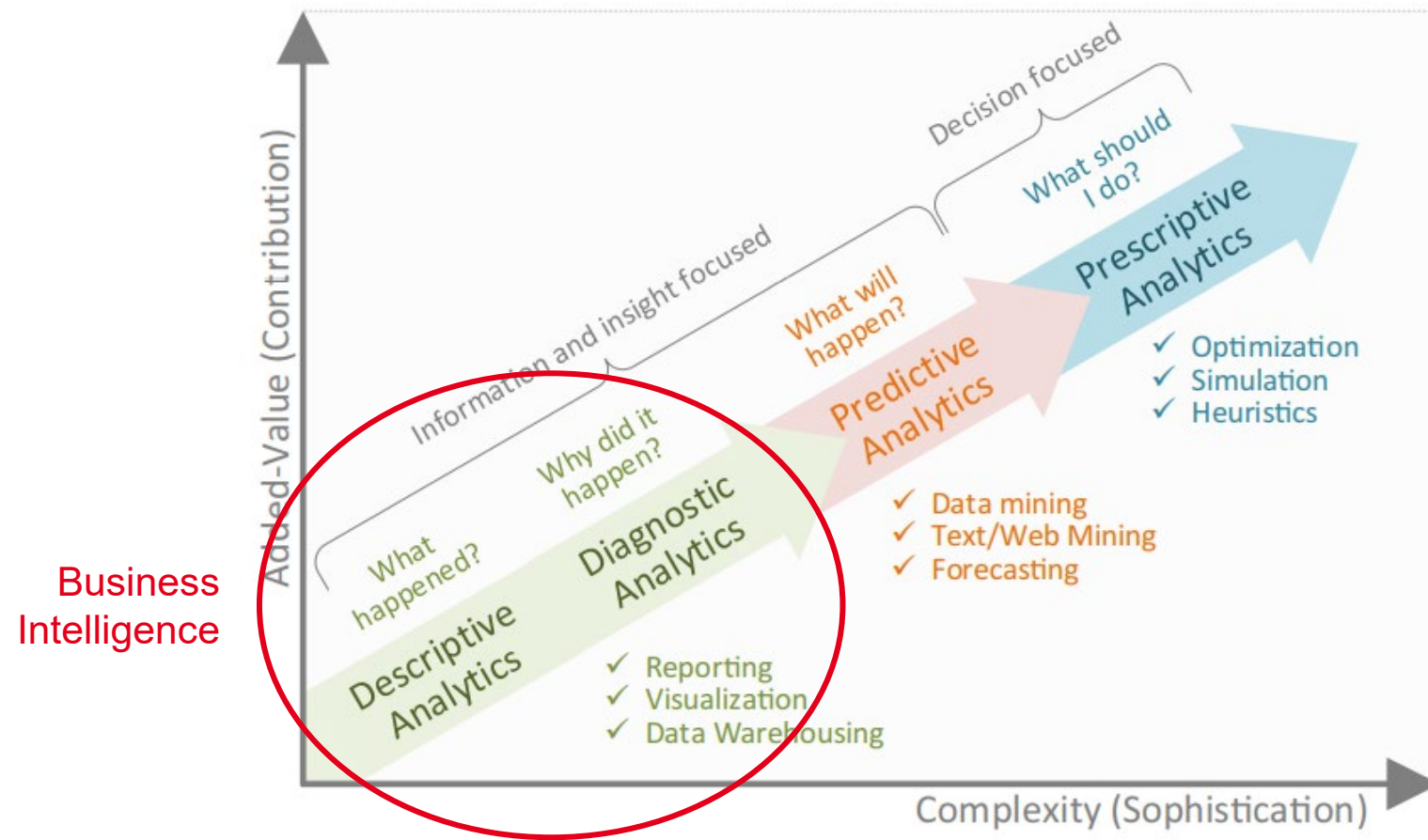
- **Datenintegration:** Unternehmen benötigten eine zentrale Plattform, um Daten aus verschiedenen Quellen zusammenzuführen und eine einheitliche Sicht auf ihre Informationen zu erhalten.
- **Optimierung der Analyse:** Operative Systeme waren nicht für umfangreiche Analysen ausgelegt, daher wurden Data Warehouses entwickelt, um komplexe Abfragen effizient zu verarbeiten.
- **Historische Datenanalyse:** Es bestand Bedarf, historische Daten zu speichern und für langfristige Analysen bereitzustellen.

Eigenschaften:

- **Strukturierte Daten:** Speichert strukturierte und bereinigte Daten, die für Abfragen optimiert sind.
- **Datenkonsistenz:** Stellt durch Transformationsprozesse eine einheitliche Datenqualität sicher.
- **Unterstützung von Business Intelligence:** Dient als Grundlage für Berichte, Dashboards und Entscheidungsfindung.

Beispiele: Amazon Redshift, Google BigQuery, Snowflake.

Stufen von Business-Analytics-Anwendungen



Delen & Zolbanin (2018)

Business Intelligence (BI)

Schlüsselfragen:

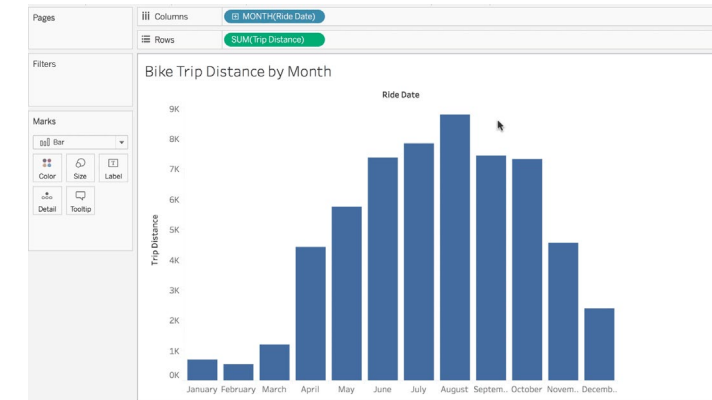
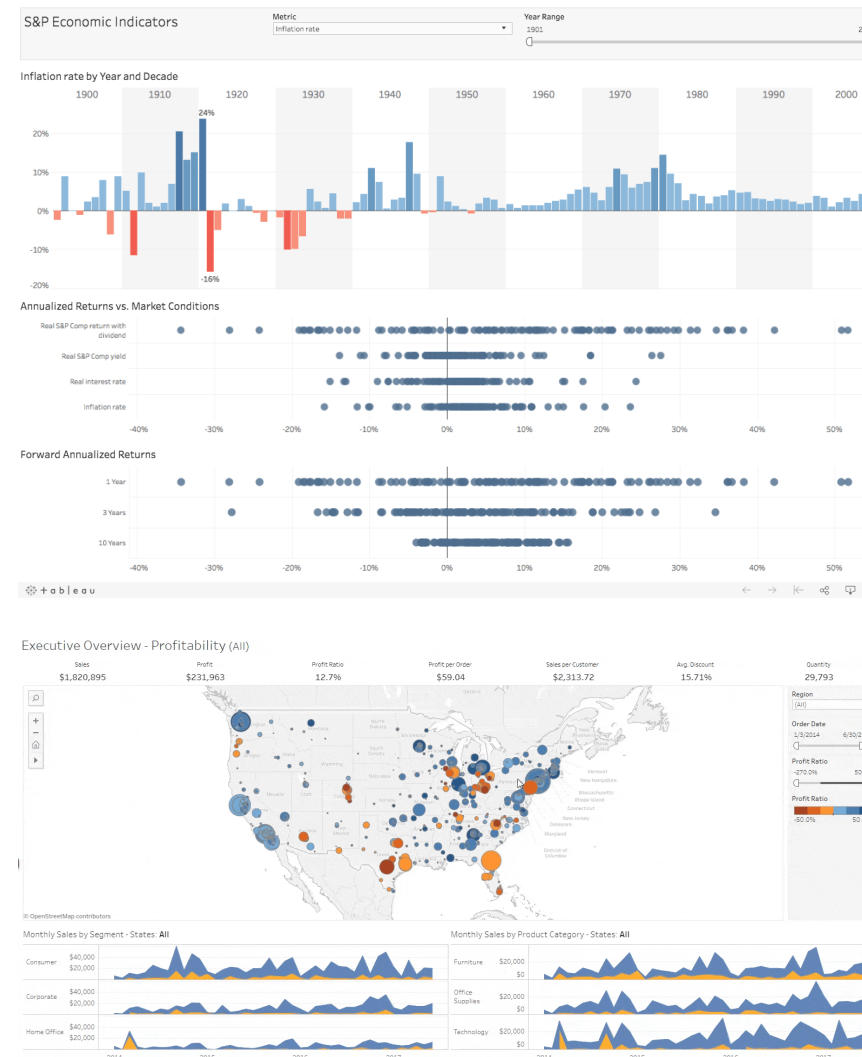
- Was ist passiert?
- Warum ist es passiert?

Voraussetzungen:

- Reichhaltige und verlässliche Datengrundlage
- Genaue Festlegung Problembeschreibung
- Genaue Lösungsbeschreibung

Use Cases:

- Visualisierung von Verkaufszahlen
- Historische Veränderungen der Kosten/Preise
- Effizienzmessungen im Kundensupport
- ... (➔ Tabellen und Dashboards)



Transaktionale Systeme vs. analytische Datenanwendungen

Transaktionale Systeme	Analytische Datenanwendungen
Fokus: operative Prozesse	Fokus: Analyse und Exploration
Viele kleine Transaktionen	Komplexe Abfragen und Auswertungen
Stark strukturierte Daten	Häufig Kombination aus strukturierten, semi-strukturierten und unstrukturierten Daten
Klassische relationale Datenbanken	Häufig alternative Datenschemata, Verwendung des relationalen Schemas aber möglich
Beispiele: Online-Shop, Banking-System	Beispiele: Datenanalyse, Dashboards, Data Apps

Datenquellen und Datenaustausch

Daten können aus sehr unterschiedlichen Quellen stammen, z.B.:

- Unternehmenssysteme (z.B. ERP, CRM, Shopsysteme)
- Web- und Cloud-Anwendungen
- öffentliche Datensätze (z.B. Open Data, Kaggle)
- Sensor- oder IoT-Daten
- Log- oder Eventdaten

Für den Austausch und die Weiterverarbeitung werden häufig standardisierte **Datenformate** verwendet. Zu diesen zählen:

- CSV
- JSON
- XML
- YAML
- Parquet

Agenda

- Organisatorisches
- Einführung: Moderne Datenanwendungen
- **Datenformate und semi-strukturierte Daten**
- NoSQL-Datenbanken
- Dokumentenorientierte Datenspeicherung mit MongoDB
- Datenverarbeitung und Datenbankbindung mit Python
- Entwicklung von Datenanwendungen mit Streamlit
- Projektarbeit: Entwicklung einer eigenen Datenanwendung
- Präsentation und Diskussion der Projekte

Offene Frage in die Runde



Strukturierte Daten

Daten liegen in einer **klar definierten, festen Struktur** vor.

Merkmale:

- festes Schema
- gleiche Struktur für alle Datensätze
- klar definierte Datentypen
- typischerweise tabellarische Organisation

Beispiel (relationale Tabelle):

Name	Alter	Ort
Anna	23	Berlin
Jonas	31	Hamburg

Semi-strukturierte Daten

Daten besitzen **eine erkennbare Struktur**, aber **kein strikt vorgegebenes Schema**.

Merkmale:

- flexible oder optionale Felder
- verschachtelte Strukturen möglich
- Datensätze können unterschiedliche Attribute besitzen

Beispiel (JSON):

```
{  
  "name": "Anna",  
  "age": 23,  
  "hobbies": ["Klettern", "Lesen"]  
}
```

Vergleich von CSV, XML und JSON

Eigenschaft	CSV	XML	JSON
Strukturtyp	strukturiert	semi-strukturiert	semi-strukturiert
Grundstruktur	tabellarisch (Zeilen und Spalten)	hierarchische Elemente (Tags)	Objekte und Arrays
Schema	meist fest (gleiche Spalten pro Zeile)	flexibel, optional mit Schema (DTD/XSD)	flexibel, optional mit Schema
Verschachtelung möglich	nein	ja	ja
Erweiterbarkeit	gering	hoch	hoch
Typische Nutzung	Datenexporte, Tabellen	Systemintegration, Dokumentformate	Web-APIs, NoSQL-Datenbanken

CSV – Comma-Separated Values

Was ist CSV?

CSV ist ein **einfaches, textbasiertes Format**, das verwendet wird, um **tabellarische Daten** in einer Datei darzustellen.

Dabei gilt:

- **Jede Zeile** in der Datei entspricht einem **Datensatz**.
- Innerhalb jeder Zeile sind die **Felder durch ein Trennzeichen** getrennt (meist ein Komma, Semikolon oder Tabulator).
- Die Datei besteht **nur aus Text**, es gibt keine Formatierung, Datentypen oder eingebettete Metadaten.

Beispiel:

Name,Alter,Ort

Anna,23,Berlin

Jonas,31,Hamburg

...

Das Format ist sehr leichtgewichtig, maschinenlesbar und menschenlesbar – und dadurch besonders weit verbreitet.

Ursprung des CSV-Formats

- Das CSV-Format hat seinen Ursprung in den **frühen Tagen der Tabellenkalkulation**, insbesondere bei **Lotus 1-2-3** und später **Microsoft Excel**.
- Ziel war es, ein **kompatibles Austauschformat** zu schaffen, mit dem Daten zwischen unterschiedlichen Programmen **ohne komplexe Konvertierungen** übertragen werden konnten.
- Da viele Programme einfache Textdateien einlesen und schreiben können, war CSV ideal für den **Export/Import zwischen Systemen**, die nicht dieselbe Software nutzen.
- **RFC 4180** beschreibt seit 2005 eine (lose) Standardisierung des Formats – allerdings gibt es **keine strenge Norm**. In der Praxis existieren viele CSV-Varianten, z. B. mit:
 - unterschiedlichen Trennzeichen (z.B. , oder ; oder |)
 - optionalen Anführungszeichen um Felder
 - Zeilenumbrüche in Feldern (nur mit bestimmten Escape-Konventionen)

Warum ist CSV bis heute relevant?

- **Einfachheit:** Keine zusätzliche Software oder Parser notwendig – CSV lässt sich mit jedem Texteditor öffnen.
- **Breite Unterstützung:** Fast jedes Datenanalyse-Tool, jede Programmiersprache und jedes Datenbanksystem kann CSV-Dateien lesen und schreiben.
- **Plattformunabhängig:** CSV-Dateien funktionieren unabhängig von Betriebssystem oder Umgebung.

Schematische Struktur von CSV-Dateien

CSV-Dateien folgen einem **sehr einfachen Aufbau**, der sich an **Tabellen** orientiert:

Spalte1,Spalte2,Spalte3

Wert1,Wert2,Wert3

Wert4,Wert5,Wert6

Merkmale:

- Die **erste Zeile** enthält häufig die **Spaltennamen** (*Header-Zeile*), ist aber optional.
- **Jede weitere Zeile** stellt einen **Datensatz** dar.
- Die Felder sind durch ein **Trennzeichen** (Separator) getrennt – meist ein **Komma**, aber auch **Semikolon** oder **Tabulator** sind gebräuchlich.
- Alle Inhalte sind **Text** – es gibt keine eingebauten Datentypen.

Beispiele für unterschiedliche Trennzeichen

Beispiel 1: Standard (Komma-separiert, „klassisches“ CSV)

```
Name,Alter,Ort  
Anna,23,Berlin  
Jonas,31,Hamburg
```

Beispiel 2: Semikolon-separiert (z. B. in Deutschland/Excel mit deutschem Sprachpaket)

```
Name;Alter;Ort  
Anna;23;Berlin  
Jonas;31;Hamburg
```

Beispiel 3: Tabulator-separiert (TSV – Tab-Separated Values)

```
Name\tAlter\tOrt  
Anna\t23\tBerlin  
Jonas\t31\tHamburg
```

Besonderheiten & Stolperfallen bei der Verwendung von CSV

- **Felder mit Trennzeichen, Zeilenumbrüchen oder Anführungszeichen** müssen oft in doppelte Anführungszeichen gesetzt werden:
 - "Max, der Zweite",27,"Köln\nAltstadt"
 - Lea Schulz,35,"Frankfurt, Main"
 - "Dr. Tim Berger",45,Hannover
- **Kein einheitlicher Standard** → Tools müssen oft auf Trennzeichen konfiguriert werden (z.B. Excel, Python csv-Modul)

XML – Extensible Markup Language

Was ist XML?

XML ist eine **textbasierte Auszeichnungssprache**, die entwickelt wurde, um **strukturierte Daten hierarchisch** darzustellen.

Der Fokus liegt dabei auf:

- **Lesbarkeit für Menschen und Maschinen**
- **Portabilität über verschiedene Systeme**
- **Darstellung komplexer Strukturen**

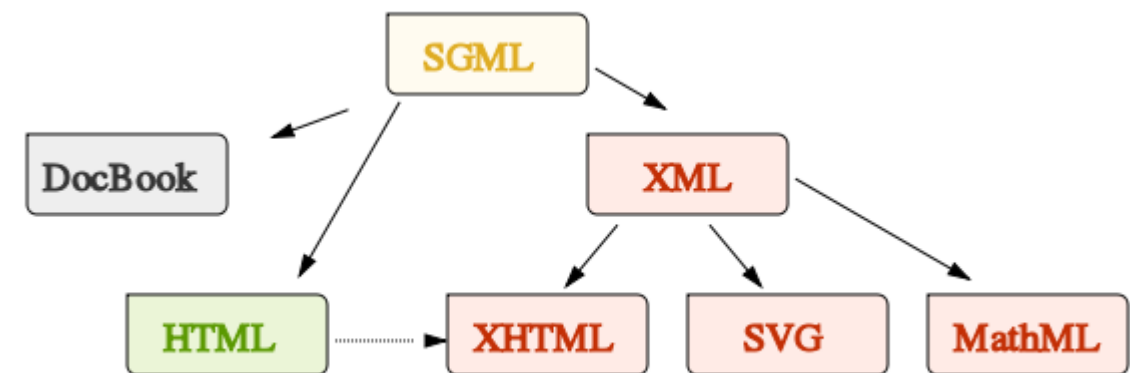
Ein XML-Dokument besteht aus sogenannten **Elementen** (Tags), die Daten umschließen und strukturieren – ähnlich wie HTML, aber inhaltlich neutral und beliebig erweiterbar.

Beispiel:

```
<person>  
  <name>Anna</name>  
  <age>23</age>  
  <city>Berlin</city>  
</person>
```

Ursprung von XML

- XML wurde in den **1990er Jahren** vom **W3C (World Wide Web Consortium)** als vereinfachte Weiterentwicklung von **SGML (Standard Generalized Markup Language)** entwickelt.
- Ziel: Eine **universelle Sprache zur Beschreibung strukturierter Daten**, die plattformübergreifend und leicht zu verarbeiten ist.
- Die erste Version erschien **1998** – seither weit verbreitet im Kontext von:
 - **Datenaustausch zwischen Anwendungen** (z.B. Webservices, SOAP)
 - **Konfigurationsdateien**
 - **Datenbanken & Dokumentenspeicher** (z.B. XML-Datenbanken)
 - **Office-Dokumenten** (z. B. .docx/.xlsx basieren intern auf XML)



<https://wiki.selfhtml.org/wiki/SGML>

Warum ist XML bis heute relevant?

Strukturiert & hierarchisch

- Elemente können verschachtelt werden → ideal für komplexe Daten
- Beispiel: Personen mit Adressen, Bücher mit Autoren etc.

Selbstbeschreibend

- Jedes Feld hat einen eindeutigen Namen
- Menschen & Maschinen erkennen sofort, was ein Wert bedeutet

Validierbar

- Mit XSD oder DTD können Struktur & Datentypen geprüft werden
- Nützlich in sicherheitskritischen oder regulierten Umgebungen

Plattform- & systemunabhängig

- Reiner Text → funktioniert auf jedem System
- Viele Standardformate basieren intern auf XML (.docx, .svg, .xlsx)

Bewährt im Datenaustausch

- Bis heute verbreitet bei Webservices (z.B. SOAP oder REST)
- Besonders relevant in der Systemintegration (z.B. Behörden, Unternehmen)

Schematische Struktur von XML

Ein XML-Dokument besteht aus:

- **Deklaration (optional):**

Gibt an, dass es sich um XML handelt und in welcher Kodierung:

```
<?xml version="1.0" encoding="UTF-8"?>
```

- **Wurzelement (Root-Element):**

Umfasst das gesamte Dokument – es darf **nur eines geben**.

```
<person> ... </person>
```

- **Elemente (Tags):**

Stellen die eigentlichen Daten dar:

```
<name>Anna</name>
```

- **Attribute (optional):**

Können zusätzliche Informationen in einem Element unterbringen:

```
<person geschlecht="weiblich">...</person>
```

Beispiele XML

- Einfaches Beispiel (Elementbasiert) mit Deklaration:

```
<?xml version="1.0" encoding="UTF-8"?>
<person>
  <name>Anna</name>
  <alter>23</alter>
  <ort>Berlin</ort>
</person>
```
- Beispiel mit Attributen und Deklaration:

```
<?xml version="1.0" encoding="UTF-8"?>
<person geschlecht="weiblich" id="42">
  <name>Anna</name>
  <ort>Berlin</ort>
</person>
```

Beispiel: Liste von Personen in XML

```
<?xml version="1.0" encoding="UTF-8"?>
<personen>
  <person id="1" geschlecht="weiblich">
    <name>Anna Müller</name>
    <alter>23</alter>
    <ort>Berlin</ort>
  </person>
  <person id="2" geschlecht="männlich">
    <name>Jonas Becker</name>
    <alter>31</alter>
    <ort>Hamburg</ort>
  </person>
  <person id="3" geschlecht="divers">
    <name>Alex Kim</name>
    <alter>27</alter>
    <ort>Köln</ort>
  </person>
</personen>
```

Validierung von XML-Dokumenten: DTD und XSD

Die Begriffe **XSD** und **DTD** beziehen sich auf die Möglichkeit, ein XML-Dokument **formell zu beschreiben und zu validieren**. Das ist besonders nützlich in professionellen Kontexten, wo Datenstrukturen klar definiert, überprüfbar und automatisiert verarbeitet werden müssen.

Begriff	Bedeutung	Ziel
DTD	Document Type Definition	Legt fest, wie ein XML-Dokument aufgebaut sein darf – ältere, einfachere Variante
XSD	XML Schema Definition	Mächtigerer, XML-basiertes Schemaformat mit Typdefinitionen und mehr Kontrolle

DTD – Document Type Definition

Merkmale:

- Definiert erlaubte **Elemente, Attribute und Struktur**
- Wird entweder **intern im XML-Dokument** oder **extern als separate Datei** eingebunden
- Kein XML-Format selbst (eigene Syntax)

Einfaches Beispiel (interne DTD):

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE person [
  <!ELEMENT person (name, alter, ort)>
  <!ELEMENT name (#PCDATA)>
  <!ELEMENT alter (#PCDATA)>
  <!ELEMENT ort (#PCDATA)>
]>
<person>
  <name>Anna</name>
  <alter>23</alter>
  <ort>Berlin</ort>
</person>
```

- #PCDATA steht für „parsed character data“ (also Text)
- Diese DTD erzwingt, dass <person> genau die drei Elemente in genau dieser Reihenfolge enthält

XSD – XML Schema Definition

Merkmale:

- Wird in **XML** geschrieben
- Unterstützt **Datentypen** (z.B. xs:integer, xs:date)
- Erlaubt feinere Kontrolle (Pflichtfelder, Reihenfolge, Längen, Wertebereiche, Typprüfungen)

Beispiel: XML mit externem XSD

```
<?xml version="1.0" encoding="UTF-8"?>
<person xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="person.xsd">
  <name>Anna</name>
  <alter>23</alter>
  <ort>Berlin</ort>
</person>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="person">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="name" type="xs:string"/>
        <xs:element name="alter" type="xs:integer"/>
        <xs:element name="ort" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Tutorials zu XML

- XML: <https://www.w3schools.com/xml/default.asp>
 - XML HOME
 - XML Introduction
 - XML How to use
 - XML Tree
 - XML Syntax
 - XML Elements
 - XML Attributes
 - XML DTD
 - XML Schema
- XML DTD: https://www.w3schools.com/xml/xml_dtd_intro.asp
 - DTD Introduction
 - DTD Building Blocks
 - DTD Elements
 - DTD Attributes
 - DTD Elements vs Attr
- XSD: https://www.w3schools.com/xml/schema_intro.asp
 - XSD Introduction
 - XSD How To
 - XSD <schema>
 - XSD Elements
 - XSD Attributes

JSON – JavaScript Object Notation

Was ist JSON?

- **JSON** ist ein **leichtgewichtiges, textbasiertes Format** zur Darstellung strukturierter Daten. Es basiert auf der Syntax von JavaScript, ist aber **programmiersprachenunabhängig** einsetzbar.
- JSON stellt Daten mithilfe von **Schlüssel-Wert-Paaren**, **Arrays** und **Objekten** dar. Es ist besonders **kompakt, lesbar** und hervorragend geeignet für die **Übertragung von Daten** zwischen Systemen – z. B. zwischen einem Webbrowser und einem Server.

Beispiel:

```
{  
  "name": "Anna",  
  "alter": 23,  
  "ort": "Berlin"  
}
```

- **Schlüssel** stehen in Anführungszeichen
- Werte können Zahlen, Strings, Booleans, Arrays, Objekte oder `null` sein
- Die Struktur ist **hierarchisch** und kann beliebig tief verschachtelt werden

Ursprung von JSON

- JSON wurde Anfang der **2000er Jahre** von **Douglas Crockford** als Antwort auf die umständliche XML-Nutzung im Web entwickelt
- Ziel: ein **einfaches, leicht verarbeitbares Format** für Webanwendungen
- JSON ist mittlerweile in **nahezu jeder Programmiersprache** unterstützt (z.B. `json` in Python, `JSON.parse()` in JavaScript)

Warum ist JSON so verbreitet?

- **Kompakt & lesbar** → ideal für Menschen und Maschinen
- **Verschachtelbar** → geeignet für komplexe Objekte und Listen
- **Standard für Web-APIs** (z.B. REST, GraphQL)
- **Einfach zu parsen** → native Unterstützung in fast jeder Sprache
- **Unabhängig vom System** → reines Textformat

Schematische Struktur von JSON

Ein JSON-Dokument besteht aus **Objekten**, **Arrays**, **Schlüssel-Wert-Paaren** und **Datentypen**. Es basiert auf der Syntax von JavaScript.

Grundelemente in JSON:

Element	Beschreibung	Beispiel
Objekt	Sammlung von Schlüssel-Wert-Paaren (geschweifte Klammern)	{ "name": "Anna" }
Array	Liste von Werten (eckige Klammern)	[23, 45, 31]
Schlüssel	Immer ein String in Anführungszeichen	"alter"
Wert	String, Zahl, Boolean, Objekt, Array oder null	"Berlin", 42, false, null

Beispiele JSON

- **Beispiel 1:** Einfaches JSON-Objekt:

```
{  
  "name": "Anna",  
  "alter": 23,  
  "ort": "Berlin"  
}
```

- **Beispiel 2:** Objekt mit Array:

```
{  
  "name": "Jonas",  
  "hobbys": ["Lesen", "Klettern", "Gitarre"]  
}
```

- **Beispiel 3:** Verschachteltes Objekt

```
{  
  "name": "Alex",  
  "adresse": {  
    "strasse": "Hauptstraße 5",  
    "stadt": "München"  
  }  
}
```

- **Beispiel 4:** Array von Objekten (Liste von Personen)

```
[  
  { "name": "Anna", "alter": 23 },  
  { "name": "Jonas", "alter": 31 },  
  { "name": "Alex", "alter": 27 }  
]
```

Besonderheiten und Stolperfallen in JSON

- Schlüssel **typischerweise** immer in doppelten Anführungszeichen (Ausnahmen: JSON5 und MongoDB-Shell; letzteres kümmert sich automatisch um Anführungszeichen)

`{ "name": "Anna" }` → RICHTIG

`{ name: "Anna" }` → NICHT ERLAUBT

- Letztes Element darf kein Komma haben

```
{  
  "name": "Anna",  
  "alter": 23  
}
```

→ KEIN KOMMA HIER

- Keine Kommentare erlaubt
- Nur bestimmte Datentypen erlaubt: String, Number, Boolean, null, Object und Array
- Zahlen: kein führendes 0, kein NaN oder Infinity

→ Bei Unsicherheit Validierung nutzen: (z.B.) <https://jsonlint.com/>

Beispiel: Liste von Personen in JSON

```
[
  {
    "id": 1,
    "name": "Anna Müller",
    "alter": 23,
    "geschlecht": "weiblich",
    "ort": "Berlin"
  },
  {
    "id": 2,
    "name": "Jonas Becker",
    "alter": 31,
    "geschlecht": "männlich",
    "ort": "Hamburg"
  },
  {
    "id": 3,
    "name": "Alex Kim",
    "alter": 27,
    "geschlecht": "divers",
    "ort": "Köln"
  }
]
```

Vergleich Liste von Personen in CSV, XML und JSON

```
id,name,alter,geschlecht,ort
1,Anna Müller,23,weiblich,Berlin
2,Jonas Becker,31,männlich,Hamburg
3,Alex Kim,27,divers,Köln
```

```
<?xml version="1.0" encoding="UTF-8"?>
<personen>
  <person id="1" geschlecht="weiblich">
    <name>Anna Müller</name>
    <alter>23</alter>
    <ort>Berlin</ort>
  </person>
  <person id="2" geschlecht="männlich">
    <name>Jonas Becker</name>
    <alter>31</alter>
    <ort>Hamburg</ort>
  </person>
  <person id="3" geschlecht="divers">
    <name>Alex Kim</name>
    <alter>27</alter>
    <ort>Köln</ort>
  </person>
</personen>
```

```
[
  {
    "id": 1,
    "name": "Anna Müller",
    "alter": 23,
    "geschlecht": "weiblich",
    "ort": "Berlin"
  },
  {
    "id": 2,
    "name": "Jonas Becker",
    "alter": 31,
    "geschlecht": "männlich",
    "ort": "Hamburg"
  },
  {
    "id": 3,
    "name": "Alex Kim",
    "alter": 27,
    "geschlecht": "divers",
    "ort": "Köln"
  }
]
```

Tutorial zu JSON

- JSON: https://www.w3schools.com/js/js_json.asp
 - JSON Intro
 - JSON vs XML
 - JSON Data Types
 - JSON Objects
 - JSON Arrays

Kurze Einordnung: YAML

YAML – Yet Another Markup Language

- YAML ist ein **textbasiertes Format zur Darstellung strukturierter Daten**. Es wird häufig für **Konfigurationsdateien** verwendet.
- Im Gegensatz zu XML oder JSON ist YAML stark auf **Lesbarkeit für Menschen** ausgelegt.

Typische Einsatzgebiete

- Konfigurationsdateien (z.B. Docker, Kubernetes)
- CI/CD-Pipelines (z.B. GitHub Actions, GitLab CI)
- Machine-Learning-Workflows
- Datenkonfiguration in Softwareprojekten

Beispiel

```
database:  
  host: localhost  
  port: 27017  
  name: analytics
```

Merkmale

- hierarchische Struktur über **Einrückung**
- sehr **kompakt und gut lesbar**
- häufige Alternative zu JSON für Konfiguration

→ YAML gehört zu den **semi-strukturierten Datenformaten**.

Kurze Einordnung: Parquet

Parquet – spaltenorientiertes Datenformat

Einordnung

- Apache Parquet ist ein **spaltenorientiertes Datenformat**, das für **analytische Datenverarbeitung** und große Datenmengen entwickelt wurde.
- Es wird häufig in **Data Lakes, Data Warehouses und Big-Data-Systemen** eingesetzt.

Prinzip der spaltenorientierten Speicherung

Name:	Anna		Jonas		Lea
Alter:	23		31		27
Stadt:	Berlin		Hamburg		München

Merkmale

- **spaltenorientierte Speicherung** → besonders effizient für analytische Abfragen
- **hohe Kompression** durch ähnliche Werte innerhalb einer Spalte
- **Schema und Datentypen werden gespeichert**
- optimiert für **große analytische Datensätze**

→ Parquet speichert **strukturierte Daten** und wird vor allem für **analytische Workloads** verwendet.

Ausblick: TOON

TOON – Token Optimized Object Notation

Einordnung

- TOON ist ein experimentelles Datenformat, das speziell für die Verarbeitung durch **Large Language Models (LLMs)** entwickelt wurde.
- Ziel ist es, strukturierte Informationen mit **möglichst wenigen Tokens** darzustellen.

Motivation

- LLMs verarbeiten Text als **Tokens**.
- Formate wie JSON enthalten viele zusätzliche Zeichen (z. B. Klammern und Anführungszeichen), die ebenfalls Tokens erzeugen.
- TOON versucht daher, **syntaktischen Overhead zu reduzieren**.

Beispiel:

```
name: Anna  
age: 23  
city: Berlin
```

Merkmale

- reduziert **Token-Anzahl** für LLM-Prompts
- einfacher, flacher Aufbau
- derzeit **kein etablierter Standard**

→ TOON gehört – ähnlich wie JSON oder YAML – zu den **semi-strukturierten Datenformaten**.

Einige Medium-Artikel zu Datenformaten

- **Beginner-Friendly Guide:** <https://medium.com/@asifazad114/why-file-formats-matter-a-beginner-friendly-guide-to-json-xml-csv-yaml-parquet-and-markdown-007b4dc7df83>
- **Understanding Data Formats:** <https://rathi-ankit.medium.com/understanding-data-formats-0851fe34c041>
- **TOON vs. JSON vs CSV:** <https://medium.com/data-science-in-your-pocket/toon-vs-json-vs-csv-9cbfbb9a93f8>
- **Bye bye JSON for LLMs:** <https://medium.com/data-science-in-your-pocket/toon-bye-bye-json-for-llms-91e4fe521b14>

TOON vs Json vs CSV

What data type to use for LLM Prompts?



Mehul Gupta

Follow

4 min read · Nov 15, 2025

Understanding Data Formats

Exploring Data Concepts Through Pizza Making!



Ankit Rathi

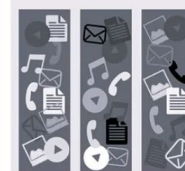
Follow

3 min read · Feb 17, 2024

Structured

1001	1001	1100
1010	0101	0110
0011	0110	0011
1100	1001	1010
0011	0101	1001
0011	1100	1001

Semi-Structured



Unstructured



Why File Formats Matter: A Beginner-Friendly Guide to JSON, XML, CSV, YAML, Parquet, and Markdown



hasifazad

Follow

3 min read · May 7, 2025

Agenda

- Organisatorisches
- Einführung: Moderne Datenanwendungen
- Datenformate und semi-strukturierte Daten
- **NoSQL-Datenbanken**
- Dokumentenorientierte Datenspeicherung mit MongoDB
- Datenverarbeitung und Datenbankbindung mit Python
- Entwicklung von Datenanwendungen mit Streamlit
- Projektarbeit: Entwicklung einer eigenen Datenanwendung
- Präsentation und Diskussion der Projekte

Offene Frage in die Runde



Was sind NoSQL-Datenbanken?

NoSQL-Datenbanken

NoSQL steht für „**Not only SQL**“ – es handelt sich also nicht um *anti-SQL*, sondern um eine **Alternative** oder **Ergänzung** zu klassischen relationalen Datenbanken.

Der Begriff bezeichnet **Datenbanksysteme**, die:

- **nicht relational** aufgebaut sind (also keine Tabellen mit festen Schemata verwenden)
- oft **flexiblere Datenmodelle** verwenden (z. B. Dokumente, Schlüssel-Wert-Paare, Graphen, Spalten)
- besonders gut mit **großen Datenmengen, verteilten Systemen** und **skalierbaren Anwendungen** umgehen können

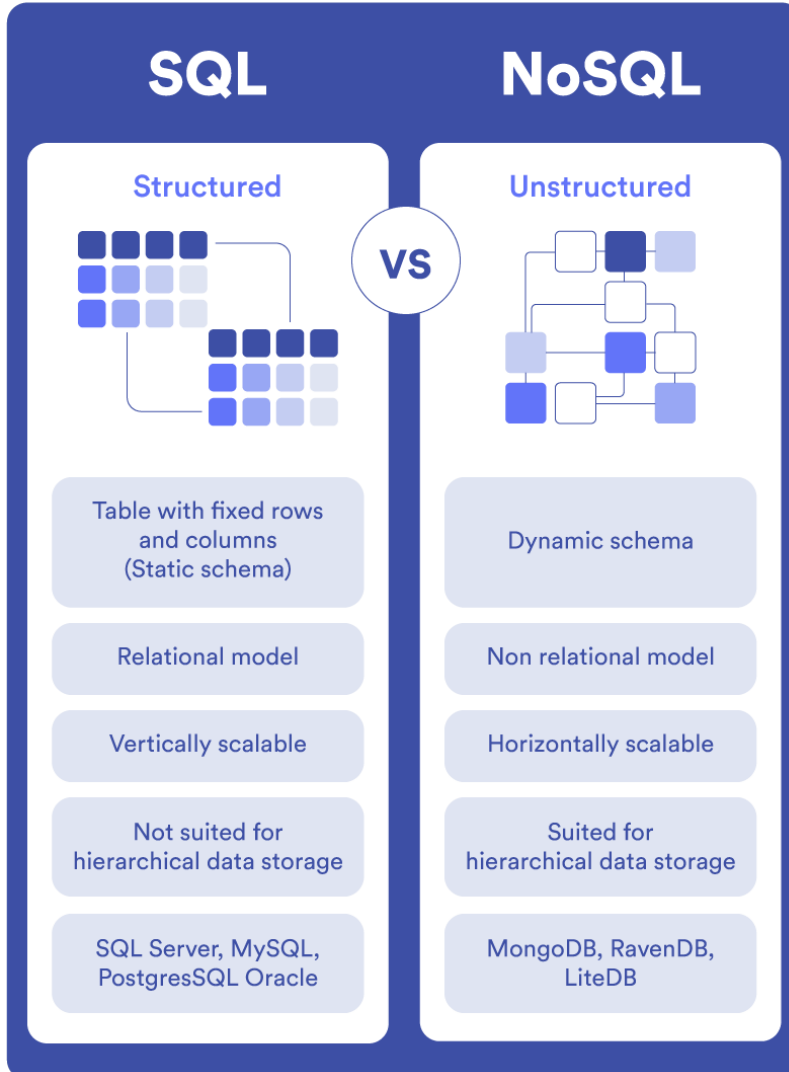
NoSQL-Systeme sind besonders nützlich, wenn:

- die Daten **nicht streng strukturiert** sind (z. B. JSON, Logs, Nutzerprofile)
- sich die Struktur häufig ändert (agile Entwicklung, Prototyping)
- extrem viele Daten gespeichert oder schnell verarbeitet werden müssen (z. B. bei Webanwendungen, IoT, Social Media)
- horizontale **Skalierung** erforderlich ist (z. B. in Cloud-Systemen oder Microservices)

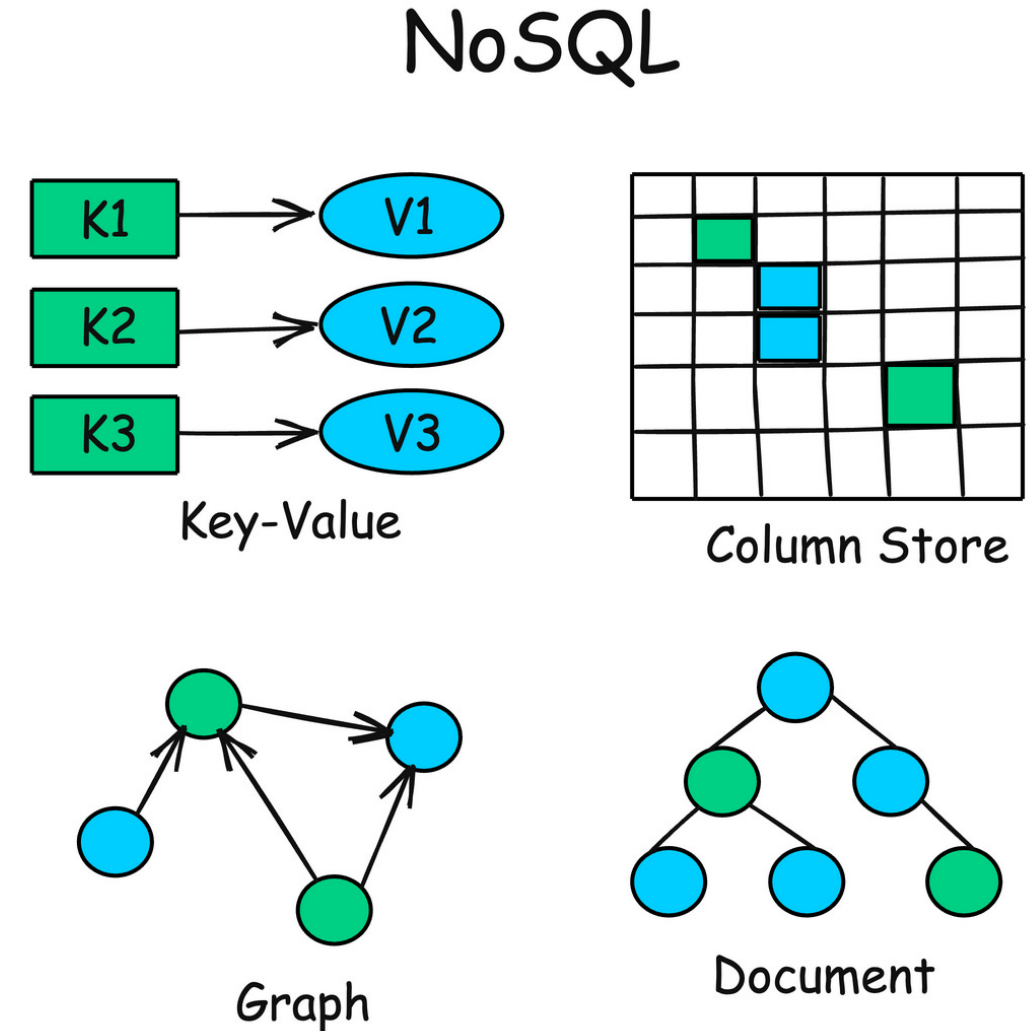
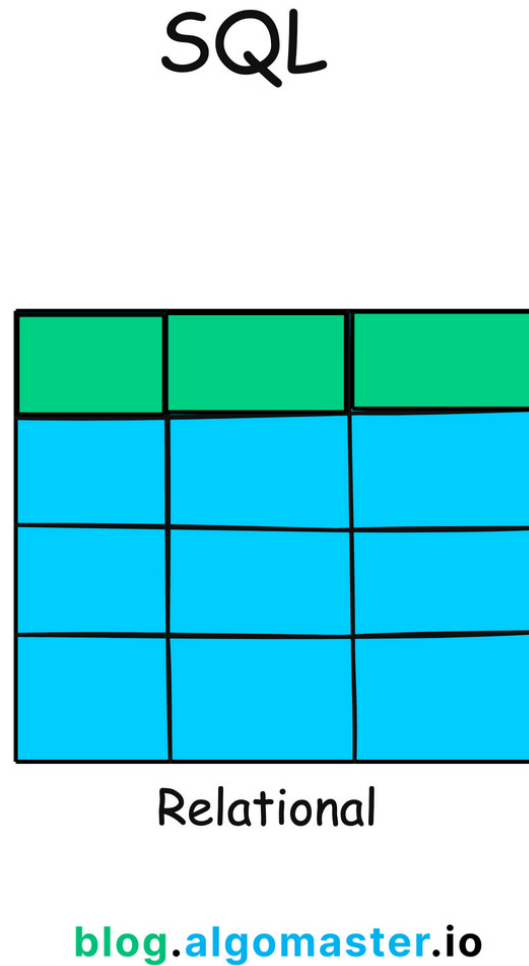
Abgrenzung zu relationalen Datenbanken

Merkmal	Relationale DB (z.B. PostgreSQL)	NoSQL (z.B. MongoDB)
Struktur	Tabellen mit festem Schema	Flexibel (z.B. Dokumente)
Datenmodell	Relational	z. B. dokumentenbasiert
Abfragen	SQL	Eigene APIs oder JSON-Queries
Joins	Voll unterstützt	Eingeschränkt oder gar nicht
Transaktionen	Stark	Eingeschränkt oder anders gelöst
Skalierung	Vertikal	Horizontal

Vergleich SQL und NoSQL



<https://pandorafms.com/blog/nosql-vs-sql-key-differences/>



Typen von NoSQL-Datenbanken

Es gibt verschiedene **Kategorien**, je nach Einsatzzweck:

1. **Dokumentenorientiert** → z.B. **MongoDB**, CouchDB
2. **Key-Value-Stores** → z.B. Redis, Amazon DynamoDB
3. **Spaltenorientiert** → z.B. Apache Cassandra, HBase
4. **Graphdatenbanken** → z.B. Neo4j, ArangoDB

Dokumentenorientiert

- Speichern Daten als **Dokumente** (meist JSON/BSON oder XML-ähnliche Strukturen).
- **Flexible Schemas** → Felder können sich zwischen Dokumenten unterscheiden.
- Ideal für **hierarchische und semi-strukturierte Daten**.
- Daten können verschachtelt werden (z.B. Arrays, Objekte).
- Abfragen oft über **Schlüssel und Attribute innerhalb der Dokumente**.

Key-Value-Stores

- Einfachstes Modell: **Schlüssel** (Key) → **Wert** (Value).
- Sehr hohe **Performance** bei einfachen Lese-/Schreiboperationen.
- Werte können beliebige Datentypen sein (Strings, JSON, BLOBs).
- Gut für **Caches, Session-Management, schnelle Lookups**.
- Abfragen sind in der Regel nur über den **Schlüssel** möglich.

Spaltenorientiert

- Daten in **Tabellen mit Zeilen und Spalten**, aber Fokus liegt auf **Spaltenfamilien** statt ganzen Zeilen.
- Eignet sich für **sehr große, verteilte Datenmengen**.
- Optimiert für **hohe Schreib- und Lese-Performance** bei großen Datenmengen.
- Abfragen oft über **bestimmte Spalten oder Spaltenbereiche**.
- Beliebt für **Big-Data-Analytics, Zeitreihendaten, Logging**.

Eine zeilenorientierte Datenbank hängt alle Datenwerte einer Zeile aneinander, es folgt die nächste Zeile usw.

```
1,Schmidt,Josef,40000;2,Müller,Maria,50000;3,Meier,Julia,44000;
```

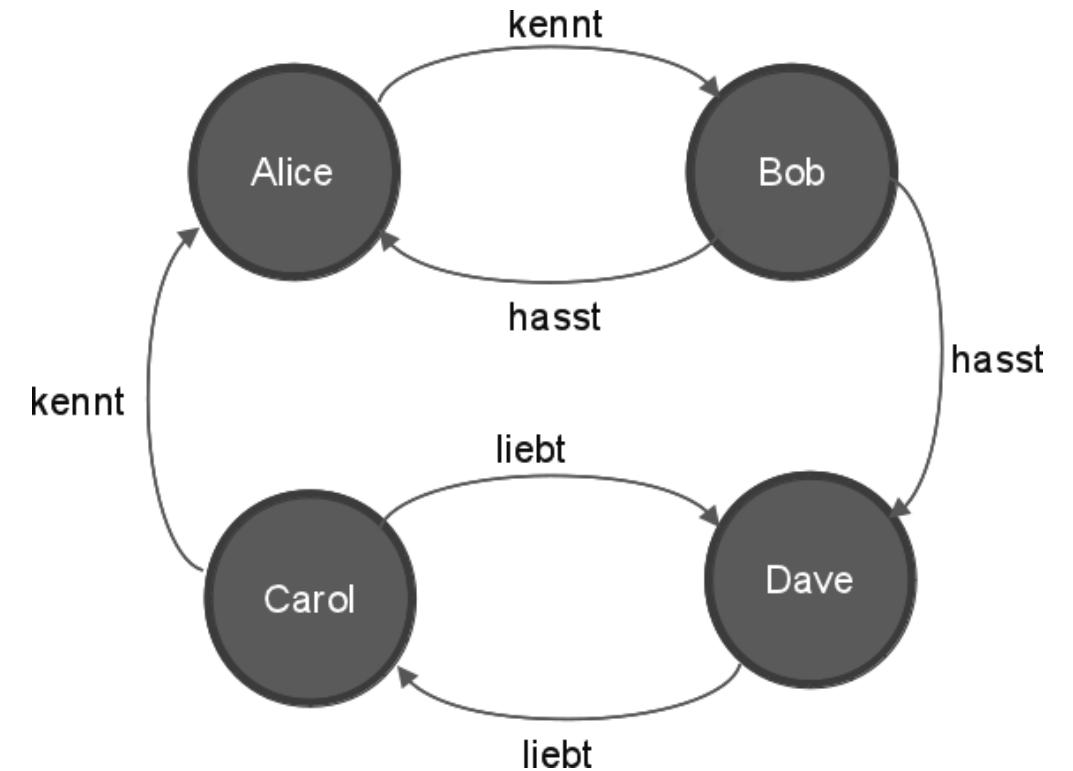
Eine spaltenorientierte Datenbank geht stattdessen Spalte für Spalte vor:

```
1,2,3;Schmidt,Müller,Meier;Josef,Maria,Julia;40000,50000,44000;
```

https://de.wikipedia.org/wiki/Spaltenorientierte_Datenbank

Graphdatenbanken

- Daten als **Knoten (Entities)** und **Kanten (Beziehungen)** gespeichert.
- Beziehungen sind **erstklassige Objekte** (nicht nur Fremdschlüssel).
- Sehr effizient für **komplexe Beziehungsabfragen** (z.B. „Kürzester Pfad“).
- Nutzen eigene **Abfragesprachen** (z.B. Cypher in Neo4j).
- Ideal für **soziale Netzwerke, Empfehlungs-Engines, Betrugserkennung**.



<https://de.wikipedia.org/wiki/Graphdatenbank>

Agenda

- Organisatorisches
- Einführung: Moderne Datenanwendungen
- Datenformate und semi-strukturierte Daten
- NoSQL-Datenbanken
- **Dokumentenorientierte Datenspeicherung mit MongoDB**
- Datenverarbeitung und Datenbankbindung mit Python
- Entwicklung von Datenanwendungen mit Streamlit
- Projektarbeit: Entwicklung einer eigenen Datenanwendung
- Präsentation und Diskussion der Projekte

Einführung in MongoDB



Was ist MongoDB?

- MongoDB ist eine **dokumentenorientierte NoSQL-Datenbank**
- Daten werden als **JSON-ähnliche Dokumente** (genauer: Binary JSON, bzw. **BSON**) gespeichert
- Entwickelt für **hohe Flexibilität, horizontale Skalierbarkeit** und **schnelle Entwicklung**
- Besonders beliebt im **Web- und App-Bereich**

Begriff	Entspricht in SQL	Beschreibung
Datenbank	Datenbank	Sammler für mehrere Collections
Collection	Tabelle	Enthält viele Dokumente gleicher Art
Dokument	Zeile (Record)	JSON-Objekt mit freier Struktur
Feld (Key)	Spalte	Schlüssel in einem Dokument

Dokument: {

```

    "name": "Anna",
    "alter": 23,
    "interessen": ["Lesen", "Musik"]
  }
```

Einsatzszenarien

Typische Einsatzszenarien

- Webanwendungen (z.B. Nutzerprofile, Foren, Kommentare)
- Content-Plattformen und CMS
- Prototypen und Startups mit sich ändernden Anforderungen
- IoT & Event-Logging
- Systeme mit vielen, schnell wachsenden Daten

MongoDB im Kontext von Data Science

Warum ist MongoDB für Data Science interessant?

- **Flexible Datenstruktur**
 - Ideal für **heterogene, halbstrukturierte** oder sich ständig ändernde Daten (z. B. Logdaten, Sensordaten, Nutzerverhalten)
 - Kein aufwendiges Aufbereiten in ein starres relationales Schema nötig
- **Nahtlose Integration mit modernen Programmiersprachen**
 - Gute Anbindung an **Python** (z.B. via pymongo) → direkte Nutzung in Jupyter Notebooks
 - JSON-artiges Datenmodell passt gut zu typischen Datenstrukturen in Pandas, NumPy etc.
- **Schnelles Prototyping & Datenzugriff**
 - Ideal für **explorative Datenanalyse**
 - Einfache Abfragen mit find(), Aggregations-Framework, Indexunterstützung
- **Skalierbarkeit bei großen Datenmengen**
 - MongoDB eignet sich für die **Vorphase der Analyse**, wenn Daten noch gesammelt, gecleant oder angereichert werden müssen

Grundlegende MongoDB-Befehle

1. Datenbank & Collection auswählen/erstellen

```
use meineDatenbank          // Wechselt zur DB (wird angelegt, falls nicht vorhanden)
db.createCollection("buecher")
```

2. Einfügen eines Dokuments

```
db.buecher.insertOne({
  titel: "MongoDB verstehen",
  autor: "Max Schröder",
  jahr: 2022
})
```

3. Mehrere Dokumente einfügen

```
db.buecher.insertMany([
  { titel: "Datenanalyse", autor: "Anna", jahr: 2021 },
  { titel: "KI kompakt", autor: "Jana", jahr: 2023 }
])
```

Grundlegende MongoDB-Befehle (cont'd)

4. Alle Dokumente anzeigen

```
db.buecher.find()
```

5. Dokumente filtern

```
db.buecher.find({ autor: "Anna" })          // einfache Abfrage
```

```
db.buecher.find({ jahr: { $gt: 2021 } })    // Jahr > 2021
```

6. Ein Dokument aktualisieren

```
db.buecher.updateOne(  
  { titel: "Datenanalyse" },  
  { $set: { jahr: 2022 } }  
)
```

7. Ein Dokument löschen

```
db.buecher.deleteOne({ titel: "KI kompakt" })
```

Vergleichsoperatoren in MongoDB

Operator	Bedeutung	Beispiel
\$gt	größer als	{ preis: { \$gt: 10 } }
\$lt	kleiner als	{ seiten: { \$lt: 300 } }
\$gte	größer oder gleich	{ jahr: { \$gte: 2020 } }
\$lte	kleiner oder gleich	{ alter: { \$lte: 18 } }
\$eq	gleich	{ sprache: { \$eq: "Deutsch" } }
\$ne	ungleich	{ autor: { \$ne: "Anna" } }

Updateoperatoren in MongoDB

Operator	Beschreibung	Beispiel
\$set	Wert eines Felds setzen oder aktualisieren	{ \$set: { jahr: 2023 } }
\$unset	Feld entfernen	{ \$unset: { seitenzahl: "" } }
\$inc	Numerischen Wert erhöhen oder verringern	{ \$inc: { seitenzahl: 50 } }
\$rename	Feld umbenennen	{ \$rename: { autor: "verfasser" } }
\$push	Wert zu einem Array hinzufügen	{ \$push: { tags: "NoSQL" } }
\$pull	Wert aus einem Array entfernen	{ \$pull: { tags: "Lehrbuch" } }
\$addToSet	Wert zu Array hinzufügen, nur wenn nicht vorhanden	{ \$addToSet: { tags: "Datenbank" } }

Beispiele:

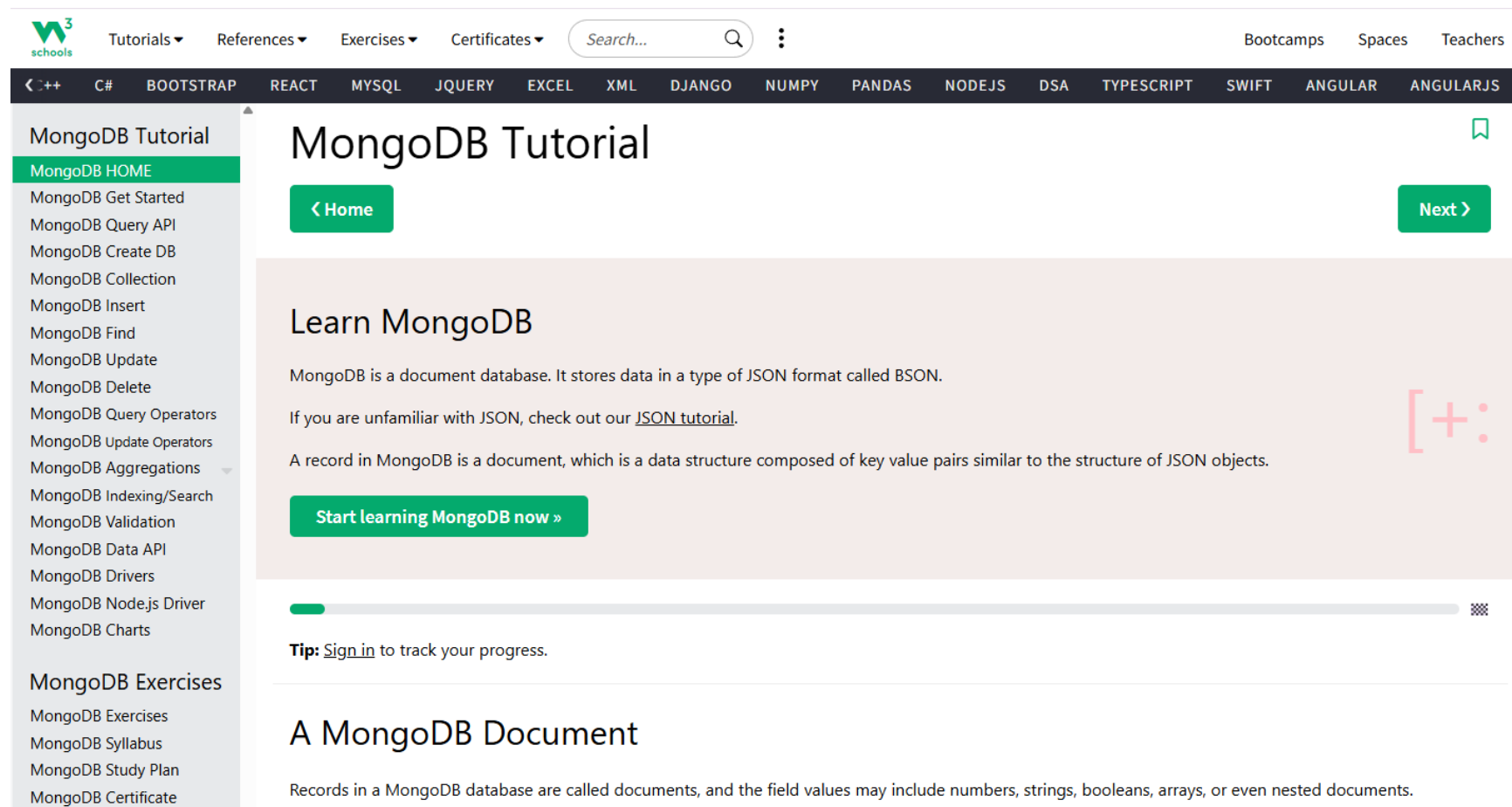
```
db.buecher.updateOne(
  { titel: "MongoDB verstehen" },
  { $set: { jahr: 2023 }, $inc: { seitenzahl: 10 } }
)
```

```
db.buecher.updateOne(
  { titel: "MongoDB verstehen" },
  {
    $inc: { seitenzahl: 20 },
    $addToSet: { tags: "Praxis" }
  }
)
```

```
db.buecher.updateOne(
  { titel: "Datenanalyse kompakt" },
  {
    $rename: { autor: "verfasser" },
    $unset: { sprache: "" }
  }
)
```

w3schools Tutorial zu MongoDB

<https://www.w3schools.com/mongodb/index.php>



The screenshot shows the w3schools MongoDB Tutorial page. The top navigation bar includes links for Tutorials, References, Exercises, and Certificates, along with a search bar and links for Bootcamps, Spaces, and Teachers. A secondary navigation bar lists various technologies like C#, BOOTSTRAP, REACT, MYSQL, JQUERY, EXCEL, XML, DJANGO, NUMPY, PANDAS, NODEJS, DSA, TYPESCRIPT, SWIFT, ANGULAR, and ANGULARJS. On the left, a sidebar menu for the MongoDB Tutorial is visible, listing topics from HOME to Charts and Exercises. The main content area is titled 'MongoDB Tutorial' and includes a 'Learn MongoDB' section with a description of MongoDB as a document database, a link to the JSON tutorial, and a 'Start learning MongoDB now' button. Below this is a progress bar and a tip to sign in. The section 'A MongoDB Document' follows, explaining that records are called documents and can contain various data types.

Agenda

- Organisatorisches
- Einführung: Moderne Datenanwendungen
- Datenformate und semi-strukturierte Daten
- NoSQL-Datenbanken
- Dokumentenorientierte Datenspeicherung mit MongoDB
- **Datenverarbeitung und Datenbankbindung mit Python**
- Entwicklung von Datenanwendungen mit Streamlit
- Projektarbeit: Entwicklung einer eigenen Datenanwendung
- Präsentation und Diskussion der Projekte

Python in dieser Veranstaltung

Für die Entwicklung der Datenanwendungen verwenden wir **Python**. Python ist eine weit verbreitete Programmiersprache für Datenanalyse, Data Science, Machine Learning, Datenverarbeitung u.v.m. → Viele Bibliotheken für Datenbanken und Datenanalyse sind direkt für Python verfügbar.

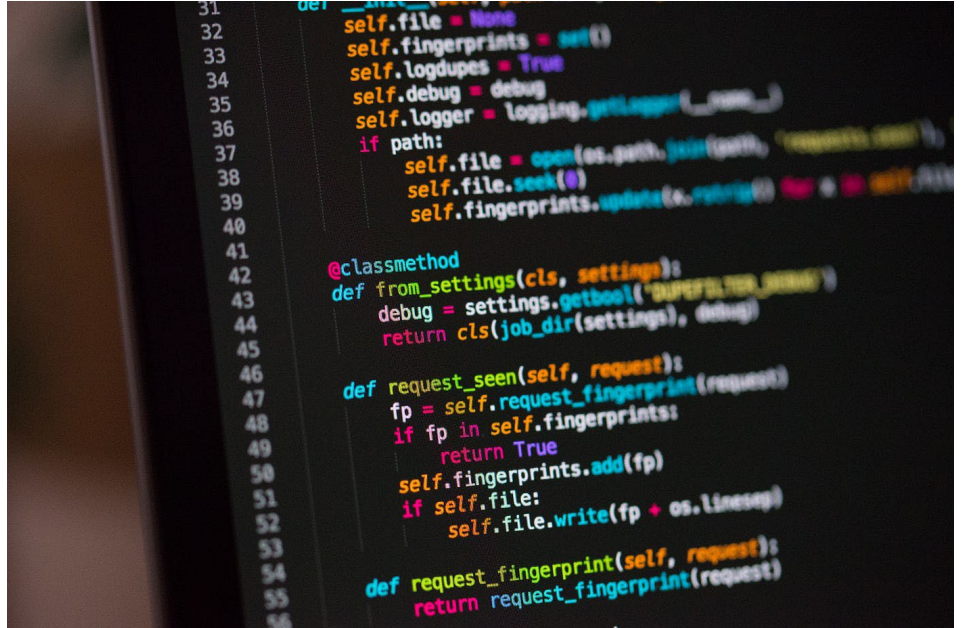
Hinweis für die Veranstaltung

In dieser Veranstaltung verwenden wir Python hauptsächlich für:

- Einlesen und Verarbeiten von Daten
- Zugriff auf Datenbanken
- Visualisierung mit Streamlit

Vorkenntnisse

- Grundlegende Programmierkenntnisse (z.B. aus Java) sind ausreichend.
- Die wichtigsten Python-Konzepte werden kurz wiederholt.



```
31 def __init__(self):
32     self.file = None
33     self.fingerprints = set()
34     self.logdupes = True
35     self.debug = debug
36     self.logger = logging.getLogger(__name__)
37     if path:
38         self.file = open(os.path.join(path, 'requests.log'),
39                         'a')
40         self.file.seek(0)
41         self.fingerprints.update(self.request_fingerprint(request))
42
43 @classmethod
44 def from_settings(cls, settings):
45     debug = settings.getbool('DEBUG', False)
46     return cls(job_dir(settings), debug)
47
48 def request_seen(self, request):
49     fp = self.request_fingerprint(request)
50     if fp in self.fingerprints:
51         return True
52     self.fingerprints.add(fp)
53     if self.file:
54         self.file.write(fp + os.linesep)
55
56 def request_fingerprint(self, request):
57     return request_fingerprint(request)
```

Wichtige Unterschiede zwischen Java und Python

Aspekt	Java	Python
Syntax	stärker formalisiert	kompakte und einfache Syntax
Typisierung	statisch typisiert	dynamisch typisiert
Programmstruktur	Klassen und Methoden erforderlich	auch einfache Skripte möglich
Blockstruktur	geschweifte Klammern {}	Einrückung (Indentation)
Variablen	Typ muss angegeben werden	Typ wird automatisch bestimmt

Beispiele: Java vs. Python

Variablendeklaration

Java

```
int age = 23;  
String name = "Anna";
```

Python

```
age = 23  
name = "Anna"
```

Beispiele: Java vs. Python (cont'd)

Bedingung (if / else)

Java

```
if (age >= 18) {  
    System.out.println("Adult");  
} else {  
    System.out.println("Minor");  
}
```

Python

```
if age >= 18:  
    print("Adult")  
else:  
    print("Minor")
```

Beispiele: Java vs. Python (cont'd)

Schleife (for)

Java

```
for (int i = 0; i < 3; i++) {  
    System.out.println(i);  
}
```

Python

```
for i in range(3):  
    print(i)
```

Beispiele: Java vs. Python (cont'd)

Array vs. Liste

Java

```
int[] numbers = {1, 2, 3};  
System.out.println(numbers[0]);
```

Python

```
numbers = [1, 2, 3]  
print(numbers[0])
```

Beispiele: Java vs. Python (cont'd)

HashMap vs. Dictionary

Java (HashMap)

```
import java.util.HashMap;

HashMap<String, Integer> ages = new
HashMap<>();

ages.put("Anna", 23);
ages.put("Jonas", 31);

System.out.println(ages.get("Anna"));
```

Python (Dictionary)

```
ages = {
    "Anna": 23,
    "Jonas": 31
}

print(ages["Anna"])
```

Wichtige Python-Bibliotheken für die Datenverarbeitung

Bibliothek	Zweck
pandas	Verarbeitung und Analyse von Daten
mysql-connector	Zugriff auf MySQL-Datenbanken
pymongo	Zugriff auf MongoDB

Datenverarbeitung mit Pandas

Einordnung

- Pandas ist eine weit verbreitete Python-Bibliothek zur **Verarbeitung und Analyse von Daten**.
- Sie wird häufig verwendet für
- das Einlesen von Datensätzen
- die Verarbeitung tabellarischer Daten
- einfache Datenanalysen

Zentrale Datenstruktur

- Die wichtigste Datenstruktur in Pandas ist der **DataFrame**.
- Ein DataFrame ist eine **tabellarische Datenstruktur**, ähnlich wie eine Tabelle in einer Datenbank oder eine Excel-Tabelle.

Pandas Dataframe

```
import pandas as pd

data = {
    "calories": [420, 380, 390],
    "duration": [50, 40, 45]
}

#load data into a DataFrame object:
df = pd.DataFrame(data)

print(df)
```

Result

	calories	duration
0	420	50
1	380	40
2	390	45

https://www.w3schools.com/python/pandas/pandas_dataframes.asp

Einlesen externer Daten mit Pandas

Einordnung

Pandas bietet Funktionen zum Einlesen von Daten aus verschiedenen Quellen.

Zu diesen zählen insb.:

- CSV-Dateien
- JSON-Dateien
- Excel-Dateien
- Parquet-Dateien

Die eingelesenen Daten werden als **DataFrame** gespeichert.

Beispiel:

```
import pandas as pd
```

```
df = pd.read_csv("data.csv")
```

```
print(df.head())
```

Erklärung

- `read_csv()` lädt eine CSV-Datei
- das Ergebnis ist ein **DataFrame**
- `head()` zeigt die ersten Zeilen des Datensatzes

Arbeiten mit DataFrames: Typische Funktionen

- Datensatz anzeigen: `df.head()`
- Anzahl der Zeilen und Spalten: `df.shape`
- Spalten auswählen: `df["age"]`
- Zeilen filtern: `df[df["age"] > 30]`
- Einfache Statistik berechnen: `df.describe()`

→ Weitere Funktionen und Beispiele u.a. unter https://pandas.pydata.org/docs/user_guide/10min.html

Zugriff auf MySQL mit Python

Einordnung

- Python kann über spezielle Bibliotheken mit relationalen Datenbanken kommunizieren.
- Diese Bibliotheken stellen eine **Schnittstelle zwischen Python und dem Datenbanksystem** bereit.

Typischer Ablauf:

- Verbindung zur Datenbank herstellen
- SQL-Abfrage ausführen
- Ergebnisse auslesen und weiterverarbeiten

Bibliothek

Für MySQL verwenden wir in dieser Veranstaltung **mysql-connector-python**. Diese Bibliothek ermöglicht es, SQL-Abfragen direkt aus Python heraus auszuführen.

Zugriff auf MySQL mit Python – Beispiel

Bibliothek installieren (mit pip): `pip install mysql-connector-python`

Beispiel: Verbindung herstellen und Daten abfragen

```
import mysql.connector

# Verbindung herstellen
conn = mysql.connector.connect(
    host="localhost",
    user="username",
    password="password",
    database="mydatabase"
)

cursor = conn.cursor()

# SQL-Abfrage ausführen
cursor.execute("SELECT * FROM users")

# Ergebnisse ausgeben
for row in cursor:
    print(row)

# Verbindung schließen
cursor.close()
conn.close()
```

Verbindung zur Datenbank aufbauen

SQL-Abfrage ausführen

Ergebnisse weiterverarbeiten

Verbindung schließen

SQL-Daten in DataFrame laden

Einordnung

SQL-Abfragen können direkt in **Pandas DataFrames** geladen werden. Dadurch lassen sich Daten aus Datenbanken leicht weiter analysieren.

```
import pandas as pd
import mysql.connector

# Verbindung herstellen
conn = mysql.connector.connect(
    host="localhost",
    user="username",
    password="password",
    database="mydatabase"
)

# SQL-Abfrage definieren
query = "SELECT * FROM users"

# Daten in DataFrame laden
df = pd.read_sql(query, conn)

print(df.head())

# Verbindung schließen
conn.close()
```

Zugriff auf MongoDB mit Python

Bibliothek installieren (mit pip): `pip install pymongo`

Beispiel:

```
from pymongo import MongoClient

# Verbindung zur MongoDB herstellen
client = MongoClient("mongodb://localhost:27017")

# Datenbank und Collection auswählen
db = client["mydatabase"]
collection = db["users"]

# Dokumente abfragen
for doc in collection.find():
    print(doc)

# Verbindung schließen
client.close()
```

Ergänzung: MongoDB-Dokumente werden in Python als **Dictionaries** dargestellt, z.B.:

```
{
    "name": "Anna",
    "age": 23,
    "city": "Berlin"
}
```

MongoDB-Daten in DataFrame laden

Einordnung

Dokumente aus MongoDB können in Python abgefragt und anschließend in einen **Pandas DataFrame** überführt werden.

```
from pymongo import MongoClient
import pandas as pd

# Verbindung herstellen
client = MongoClient("mongodb://localhost:27017")

db = client["mydatabase"]
collection = db["users"]

# Dokumente abrufen
data = list(collection.find())

# DataFrame erstellen
df = pd.DataFrame(data)

print(df.head())

# Verbindung schließen
client.close()
```

Datenbankoperationen mit Python

Einordnung

Mit Python-Bibliotheken wie **mysql.connector** und **pymongo** können Datenbanken vollständig aus Programmen heraus gesteuert werden.

Typische Operationen sind:

Operation	Bedeutung
Create	neue Daten einfügen
Read	Daten abfragen
Update	vorhandene Daten ändern
Delete	Daten löschen

Beispiele (Read)

- **MySQL:** `cursor.execute("SELECT * FROM users")`
- **MongoDB:** `collection.find()`

W3schools Tutorials

- **Python Grundlagen:** <https://www.w3schools.com/python/default.asp>
- **Pandas:** <https://www.w3schools.com/python/pandas/default.asp>
- **Python MySQL:** https://www.w3schools.com/python/python_mysql_getstarted.asp
- **Python MongoDB:** https://www.w3schools.com/python/python_mongodb_getstarted.asp

Agenda

- Organisatorisches
- Einführung: Moderne Datenanwendungen
- Datenformate und semi-strukturierte Daten
- NoSQL-Datenbanken
- Dokumentenorientierte Datenspeicherung mit MongoDB
- Datenverarbeitung und Datenbankbindung mit Python
- **Entwicklung von Datenanwendungen mit Streamlit**
- Projektarbeit: Entwicklung einer eigenen Datenanwendung
- Präsentation und Diskussion der Projekte

Streamlit

*“Streamlit is a **free and open-source framework** to rapidly build and share beautiful **machine learning and data science web apps**. It is a **Python-based library** specifically designed for machine learning engineers. Data scientists or machine learning engineers are not web developers and they're not interested in spending weeks learning to use these frameworks to build web apps. Instead, they want a tool that is **easier to learn and to use**, as long as it can **display data and collect needed parameters for modeling**. Streamlit allows you to create a stunning-looking application with only **a few lines of code**” (datacamp).*



Einführung Streamlit

Was ist Streamlit?

Streamlit ist ein **Open-Source-Python-Framework**, das es ermöglicht, interaktive Webanwendungen für Datenanalysen und maschinelles Lernen mit minimalem Codeaufwand zu erstellen.

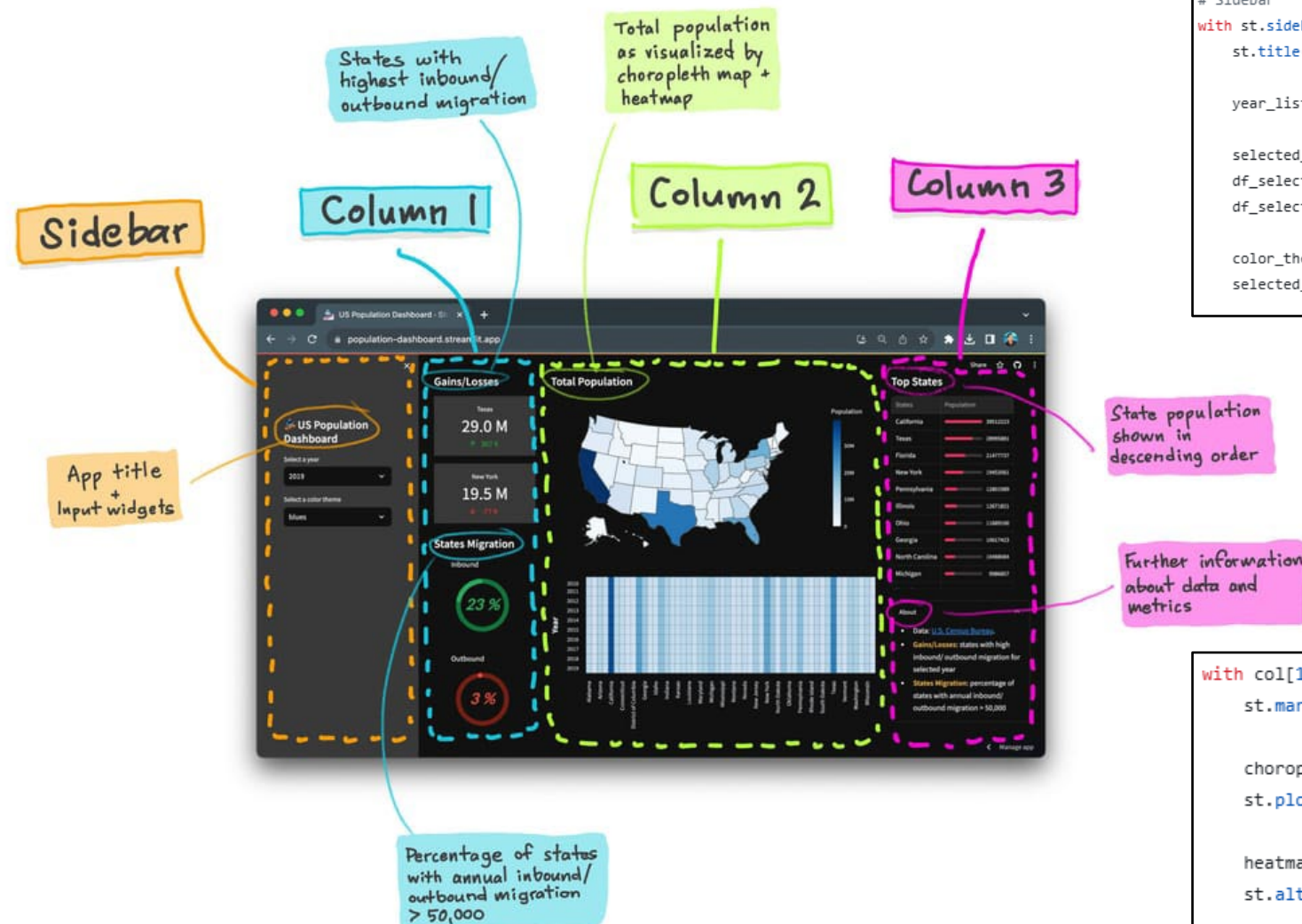
Entwicklung und Veröffentlichung

Entwickelt im Jahr 2018 von Adrien Treuille, Amanda Kelly und Thiago Teixeira, wurde die erste Version von Streamlit im Oktober 2019 veröffentlicht und gewann schnell an Beliebtheit, insb. unter Data-Scientists und Data-Engineers.

Einsatzmöglichkeiten:

- **Interaktive Dashboards:** Erstellung von Dashboards zur Visualisierung von Daten in Echtzeit (d.h. Einsatz als BI-Tool möglich).
- **Machine-Learning-Modelle:** Bereitstellung und Visualisierung von ML-Modellen für Endnutzer.
- **Datenexploration:** Schnelle und einfache Datenexploration mit interaktiven Widgets.
- **Prototyping:** Rasche Entwicklung und Testung von Datenanwendungen und -ideen.

Beispiel Streamlit



```
# Sidebar
with st.sidebar:
    st.title('🇺🇸 US Population Dashboard')

    year_list = list(df_resaped.year.unique()[::-1])

    selected_year = st.selectbox('Select a year', year_list)
    df_selected_year = df_resaped[df_resaped.year == selected_year]
    df_selected_year_sorted = df_selected_year.sort_values(by="population", ascending=False)

    color_theme_list = ['blues', 'cividis', 'greens', 'inferno', 'magma', 'plasma', 'reds', 'rainbow', 'turbo', 'viridis']
    selected_color_theme = st.selectbox('Select a color theme', color_theme_list)
```

```
with col[1]:
    st.markdown('#### Total Population')

    choropleth = make_choropleth(df_selected_year, 'states_code', 'population', selected_color_theme)
    st.plotly_chart(choropleth, use_container_width=True)

    heatmap = make_heatmap(df_resaped, 'year', 'states', 'population', selected_color_theme)
    st.altair_chart(heatmap, use_container_width=True)
```

<https://blog.streamlit.io/crafting-a-dashboard-app-in-python-using-streamlit/> und https://github.com/dataprofessor/population-dashboard/blob/master/streamlit_app.py

Einige wichtige Streamlit-Komponenten

Layout und Anzeige

- `st.title()`
- `st.header()`
- `st.write()`
- `st.dataframe()`
- `st.table()`
- `st.image()`

Visualisierung

- `st.line_chart()`
- `st.bar_chart()`
- `st.map()`

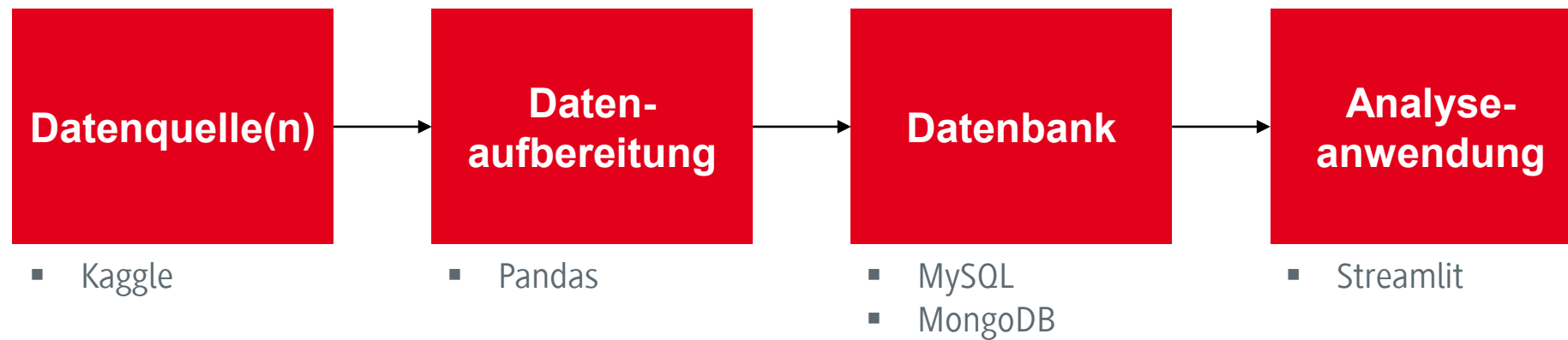
Interaktive Widgets

- `st.button()`
- `st.slider()`
- `st.selectbox()`
- `st.text_input()`
- `st.checkbox()`

Layout

- `st.sidebar`
- `st.columns`

Architektur einer analytischen Datenanwendungen in dieser Veranstaltung



Streamlit: Getting started

- Installieren Sie **Streamlit** und machen Sie sich mit den **Funktionen** der Library vertraut.
- Hier einige hilfreiche Links:
 - Streamlit-Dokumentation – Get started (sehr hilfreich!): <https://docs.streamlit.io/get-started>
 - Dashboard-Tutorial: <https://blog.streamlit.io/crafting-a-dashboard-app-in-python-using-streamlit/>
 - Zugehöriger Code: https://github.com/dataprofessor/population-dashboard/blob/master/streamlit_app.py
 - Zugehöriges Frontend: <https://population-dashboard.streamlit.app/?ref=blog.streamlit.io>
 - Ganz einfache Streamlit-App: <https://medium.com/@chaitanyasirivuri/building-your-first-streamlit-app-a-step-by-step-tutorial-e058d5dfe5f4>
 - Streamlit-Tutorial von datacamp: <https://www.datacamp.com/tutorial/streamlit>
 - Etwas Inspiration: <https://github.com/jrieke/best-of-streamlit>
- Falls Sie Probleme mit der **lokalen Installation** haben sollten, nutzen Sie **Google Colab**, siehe z.B. hier: <https://discuss.streamlit.io/t/how-to-launch-streamlit-app-from-google-colab-notebook/42399>
- Lassen Sie sich gerne so viel wie nötig von einer KI Ihrer Wahl helfen.

Agenda

- Organisatorisches
- Einführung: Moderne Datenanwendungen
- Datenformate und semi-strukturierte Daten
- NoSQL-Datenbanken
- Dokumentenorientierte Datenspeicherung mit MongoDB
- Datenverarbeitung und Datenbankbindung mit Python
- Entwicklung von Datenanwendungen mit Streamlit
- **Projektarbeit: Entwicklung einer eigenen Datenanwendung**
- Präsentation und Diskussion der Projekte

MySQL-Datenbank über phpMyAdmin nutzen (neue User!)

- Link: <https://wi-web.heilbronn.dhbw.de/phpmyadmin/> oder <http://10.150.16.50/phpmyadmin/>
- Initialpasswort für alle User: **WI2026!!nitPwd**
- User: **WI24A2_3_DB_User** + [ihre Nummer] → d.h. z.B. **WI24A2_3_DB_User5**, **WI24A2_3_DB_User22**, usw.
- **Achtung:** Datenbanken immer mit User-Präfix anlegen, d.h. z.B. **WI24A2_3_DB_User2_DB1**

Datenbankzugriff MongoDB wird nachgereicht

- Per Moodle

Projektarbeit: Entwicklung einer Datenanwendung

Aufgabe

Entwickeln Sie eine **interaktive Datenanwendung mit Streamlit**, die einen öffentlich verfügbaren Datensatz analysiert und visualisiert.

Mögliche Datenquellen:

- Kaggle
- Open Data Portale
- öffentliche APIs
- andere frei verfügbare Datensätze

Ziel der Anwendung

Die Anwendung soll es ermöglichen,

- Daten **interaktiv zu erkunden**
- **Visualisierungen** zu erzeugen
- interessante **Zusammenhänge** im Datensatz sichtbar zu machen.

Mindestanforderungen an die Datenanwendung

Die entwickelte Anwendung sollte folgende Elemente enthalten:

- **Datenquelle:** Einlesen eines externen Datensatzes (z.B. **Kaggle**, Open Data, API)
- **Datenverarbeitung:** Verarbeitung oder Transformation der Daten mit **Pandas**
- **Datenbanken:** Die Daten sollen in **zwei unterschiedlichen Datenbanksystemen gespeichert werden:**
 - **MySQL** (relationale Datenbank)
 - **MongoDB** (NoSQL-Datenbank)
 - Die Anwendung soll Daten aus diesen Datenbanken **lesen und verwenden**.
- **Interaktive Anwendung:** Entwicklung einer **Streamlit-App mit** mindestens einem **interaktiven Widget** (z.B. Filter oder Auswahl)
- **Visualisierung:** mindestens eine **grafische Darstellung** der Daten

Präsentation der Ergebnisse

Am Ende der Veranstaltung präsentieren Sie Ihre Anwendung kurz.

Inhalt der Präsentation

- Vorstellung des Datensatzes
- Ziel der Anwendung
- kurze Demonstration der App
- wichtigste Erkenntnisse aus der Datenanalyse

Dauer: ca. 5 Minuten pro Person

Prüfungsleistung: Schriftliches Assignment

Zusätzlich zur Anwendung erstellen Sie einen **Projektbericht (5–10 Seiten)**.

Der Bericht sollte folgende Punkte enthalten:

- Beschreibung des Datensatzes
- Ziel der Anwendung
- Aufbau und Funktionsweise der Anwendung
- verwendete Technologien (Python, Pandas, Streamlit)
- zentrale Ergebnisse oder Erkenntnisse

Abgabedeadline: TBD

**VIELEN DANK FÜR
IHRE AUFMERKSAMKEIT
UND MITARBEIT**

